

ENTERPRISE ARCHITECTURE BLUEPRINT

CAREERFORGE PRO

ATS Resume Builder & AI Job Optimization Platform

Comprehensive Frontend System Design Specification

Document Version:	3.0.0-Release (Enterprise Grade)
Target Audience:	Chief Architect, Principal Engineers, Senior Product Designers, Design System Review Board, Technical Architecture Committee
Core Tech Stack:	React.js, Vite, JavaScript, Redux Toolkit, React Router DOM, Axios, Tailwind CSS, React Hook Form, Sonner
Author/System:	Principal Product Designer & Staff UX Architect
Classification:	Confidential / Proprietary Engineering Blueprint

SECTION 1 — EXECUTIVE SUMMARY

1.1 Product Vision

CareerForge Pro is a next-generation SaaS ecosystem engineered to bridge the qualitative gap between high-caliber job candidates and corporate Applicant Tracking Systems (ATS). Traditional resume preparation focuses heavily on aesthetic layouts that often fail modern tokenizers and algorithmic indexers. CareerForge Pro solves this by decoupling raw data entry from template generation, using a client-side real-time structural parsing model that guarantees document compliance with legacy and modern parsers alike.

1.2 Business Objectives

- **SaaS Market Penetration:** Capture recurring subscription revenue across multi-tier accounts by providing distinct utility barriers (AI optimization quotas, premium templates).
- **Unparalleled Performance:** Guarantee sub-200ms document rendering performance during continuous client-side typing and background keyword density parsing.
- **Strategic Differentiation:** Combine high-fidelity PDF output rendering with real-time text optimization and feedback streams.

1.3 User Personas

The Active Job Seeker: Demands quick template shifting, contextual alignment with target jobs, and real-time score indicators to update resumes per job posting.

The High-Velocity Applicant: Requires mass management of tailored variations, persistent pipeline history logs, and instant cover letter generation.

The Platform Administrator: Needs to monitor revenue metrics, update template layouts, track system performance metrics, and review user status databases.

1.4 Functional & Non-Functional Scope

The core application handles multi-pane workspace layout structures, reactive text fields, background ATS score assessment, and asynchronous cover letter workflows. Out of scope are direct backend template compile pipelines; all rendering layers compile via standard web browser layout engines to maintain high fidelity. Non-functional goals target a initial bundle file size below 250KB via route splitting and explicit compliance with WCAG 2.2 AA accessibility requirements.

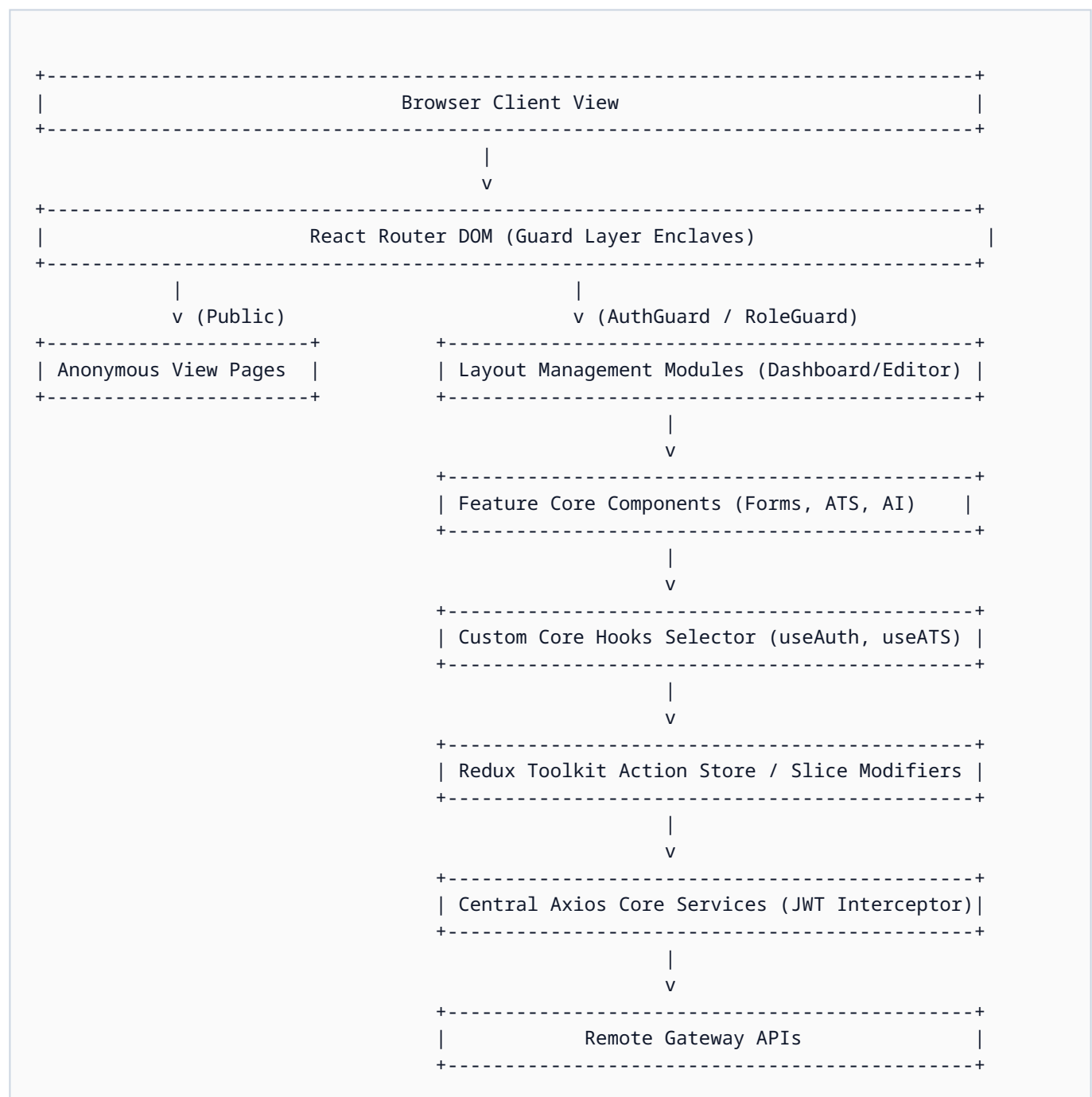
1.5 SaaS Monetization & Competitive Positioning

Monetization follows an escrowed Subscription Framework with three tiers: Free (1 base template, basic scoring), Pro (multi-document spaces, deep keyword suggestions, asynchronous AI bullet rewriter), and Enterprise (unlimited variants, automated pipeline matching, bulk cover letter processing). Unlike legacy word processors, CareerForge Pro prioritizes structural data schema definitions tailored for automated corporate parsing infrastructure.

SECTION 2 — COMPLETE FRONTEND ARCHITECTURE

The application architecture employs a deterministic state machine pattern built on React.js and Redux Toolkit. Presentation components are separated from operational side-effect triggers to keep execution paths predictable and maintain clear boundaries across the system.

2.1 Frontend Architecture Flow Map



2.2 Domain Subsystem Flows

Authentication System: Validated entry payloads trigger endpoints in the authorization service. Successful actions extract JWT parameters, populate slice stores, set authorization variables to active, and route to default dash interfaces. Failures trigger alerts via Sonner toast notifications.

ATS Parser Subsystem: Component interactions pass values through a 500ms client debounce filter. The system extracts string data into array sets, cross-references word frequencies against job definitions, computes score metrics, updates data slices, and updates score layout displays.

AI Optimization Subsystem: Component requests invoke asynchronous middleware actions. Processing triggers pass arguments to remote endpoints, which return optimized text suggestions. The subsystem updates slice states, stops spinner icons, and updates preview tracks.

SECTION 3 — COMPLETE ROUTE ARCHITECTURE

The application's route landscape uses layout grouping strategies and security wrappers managed by React Router DOM to protect system boundaries.

Route Path Target	Functional Purpose	Access Type	Layout / Components Used	Triggered API Endpoints	State Slice Dependencies
/	Public Portal Landing Terminal	Public	Home, Navbar, Footer Layouts	None	authSlice
/features	Product Capability Index	Public	Features, Navbar, Footer	None	None
/pricing	Tier Presentation Matrix	Public	Pricing, PricingCard Layouts	GET /api/v1/plans	paymentSlice
/auth/login	Security Session Access Portal	Public	Login, Input, Button Fields	POST /api/v1/auth/login	authSlice
/auth/register	Tenant Workspace Provisioning	Public	Register, Input, Button Primitives	POST /api/v1/auth/register	authSlice
/dashboard	User Metric Portfolio Hub	AuthGuard	Dashboard, DashboardLayout, Sidebar	GET /api/v1/dashboard/summary	authSlice, resumeSlice
/my-resumes	Portfolio Document Ledger	AuthGuard	MyResumes, DashboardLayout	GET /api/v1/resumes	resumeSlice
/workspace/:resumeId	High-Density Split Workspace	AuthGuard	ResumeEditor, ResumeBuilder, Preview	GET /api/v1/resumes/:id	resumeSlice, atsSlice, aiSlice
/admin	System Operational Command Hub	RoleGuard	AdminDashboard, RevenueChart	GET /api/v1/admin/metrics	authSlice, paymentSlice

SECTION 4 — COMPLETE SCREEN INVENTORY

4.1 SCR-01: Public Portal Landing Page

Purpose: High-conversion entry portal showing platform core capabilities. Entry is handled through direct root URL navigation, and exit occurs via login, register, or pricing choices.

Layout Topology: Organized using vertical and horizontal stacks. Top navigation bars use space-between properties to position branding text away from navigation elements. The main hero section uses vertical alignment structures with a 32px gap between content objects. The primary content cards use a 3-column structural row grid layout.

UX States: Content sections show subtle pulsing loading block animations while data fetches. Active cards dynamically adjust their borders to accent colors when hovered over. Input errors generate clear toast alerts.

4.2 SCR-05: Unified Multi-Pane Creation Workspace

Purpose: High-density side-by-side interface where candidates alter content data and monitor real-time parser output formatting scores. Entry points navigate through dashboard document records, exiting via the top navigation back triggers.

Layout Topology: Horizontal layout arrangement covering the full container height, split into structural left and right halves. The left editing view is constrained to a fixed 640px width with isolated vertical scrolling rules. The right preview view uses a fluid column layout, centering an A4 template simulation framework styled with subtle drop-shadows.

Validation & Interactions: Text box updates change values inside state slices, updating text layout definitions in the preview window. The system checks strings for length constraints and missing words, altering input borders if input values fail validation metrics.

SECTION 5 — COMPLETE FIGMA DESIGN SYSTEM

5.1 Variable Tokens Matrix

The layout color architecture uses primitive variables that map directly to semantic utility states, maintaining styling consistency across both color modes.

- `primitive.blue.600`: #2563eb | `primitive.blue.700`: #1d4ed8
- `primitive.slate.50`: #f8fafc | `primitive.slate.900`: #0f172a | `primitive.slate.950`: #020617
- `semantic.color.background.canvas`: Light: `primitive.slate.50` | Dark: `primitive.slate.950`
- `semantic.color.background.surface`: Light: #ffffff | Dark: `primitive.slate.900`

- `semantic.color.text.primary`: Light: `primitive.slate.900` | Dark: `primitive.slate.50`
- `semantic.color.brand.action`: Light: `primitive.blue.600` | Dark: `primitive.blue.500`

5.2 Typography Scale Engine

The design system uses `Inter` for standard interface text and optional `Lora` elements for simulated template canvases.

- `font/display-hero`: Size: 36px | Weight: 700 | Line-Height: 44px | Spacing: -0.02em
- `font/section-title`: Size: 24px | Weight: 700 | Line-Height: 32px | Spacing: -0.01em
- `font/body-bold`: Size: 14px | Weight: 600 | Line-Height: 20px | Spacing: 0
- `font/body-regular`: Size: 14px | Weight: 400 | Line-Height: 20px | Spacing: 0

SECTION 6 — COMPONENT LIBRARY SPECIFICATION

6.1 Primary Action Component (`Button.jsx`)

Inputs: `label`, `onClick`, `isDisabled`, `isLoading`. **Outputs:** Pointer action callbacks. **Interactions:** The standard state shows background brand colors with a bold white text style. Hover states apply dark accent tones. Focused elements draw a 2px outer bounding ring. Disabled items use muted surface colors, blocking pointer actions. Loading states hide text labels to show centered rotating inline SVG spinners.

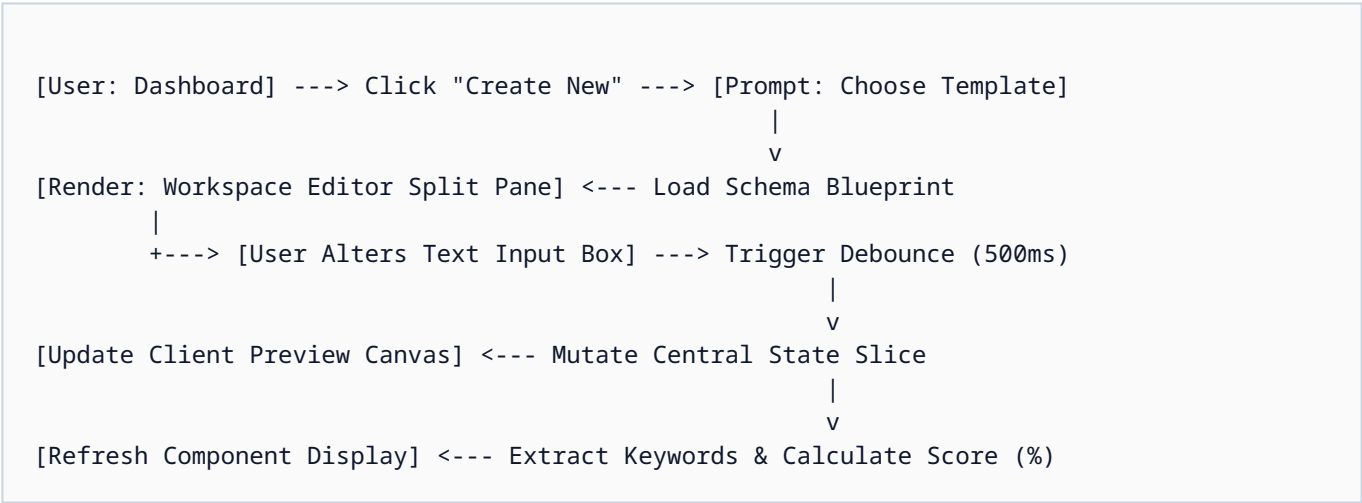
6.2 Structured Form Input Component (`Input.jsx`)

Inputs: `name`, `type`, `placeholder`, `value`, `error message`. **Interactions:** Normal states display thin gray outlines over standard surface canvases. Active focus changes outline values to 2px brand colors. Validation failures change outline boundaries to destructive red alerts, rendering error text strings below the field containers.

SECTION 7 — USER FLOW DOCUMENTATION

The system routes user interactions through deterministic sequences to prevent data conflicts during background processing tasks.

7.1 Resume Creation & Real-Time Optimization Flow



SECTION 8 — API CONTRACT MAPPING

Asynchronous operations use structured JSON formatting schemas to verify network payload integrity.

Service Target Interface	HTTP Routing Path	JSON Payload Request Structure	Success Response Payload (200 OK)	Error Action Handling Protocols
auth.service.js	POST /api/v1/auth/login	{ "email": "str", "password": "str" }	{ "success": true, "token": "JWT_KEY", "user": {} }	401 Unauthorized: Displays authentication failure alert via top-level Sonner toasts.
resume.service.js	GET /api/v1/resumes/:id	None	{ "id": "str", "sections": { "experience": [] } }	404 Not Found: Redirects application router paths back onto user dashboards.
ats.service.js	POST /api/v1/ats/analyze	{ "resumeId": "str", "jobDesc": "str" }	{ "score": 84, "missingKeywords": ["str"] }	422 Unprocessable: Flags layout field blocks needing content modification updates.

SECTION 9 — REDUX STATE ARCHITECTURE

The front-end runtime client acts as a predictable data framework, split into separate state fields inside the Redux state repository.

- **authSlice:** Tracks user authentication statuses, identity data payloads, authorization keys, and network connection states.
- **resumeSlice:** Manages document configuration objects, tracking history arrays to handle structural change rollbacks.
- **atsSlice:** Holds current compliance metrics, tracking matched phrase vectors along with missing key token sets.

SECTION 10 — RESPONSIVE DESIGN BLUEPRINT

Layout configurations adjust their structure across target device breakpoints to keep multi-pane tools usable on smaller screens.

- **Desktop Layout Wide (1440px):** Displays side-by-side split screen work areas with isolated inner vertical scrollbars.
- **Tablet Workspace Configuration (768px):** Sidenav paths collapse into icon shortcut columns. Main content fields stack vertically above document template preview blocks.
- **Mobile Fluid Workspace (375px):** Menus fold away into sliding overlay drawers. Content sections hide behind tab triggers, allowing creators to toggle viewports between data forms and simulated template outputs.

SECTION 11 — ACCESSIBILITY ARCHITECTURE

The application implements programmatic code checking helpers to ensure strict compliance with WCAG 2.2 AA accessibility requirements.

- **Keyboard Navigation Patterns:** Modal windows lock focus sequences inside active content components, preventing focus leaks to background elements until frames are closed.
- **Programmatic Metadata Identification:** Input items link to labels via explicit identifier fields, while dynamic score objects use `aria-live="polite"` attributes to announce updates cleanly to screen readers.
- **Color Selection Thresholds:** Canvas color combinations maintain a 4.5:1 text-to-background contrast ratio across both layout view choices.

SECTION 12 — PERFORMANCE ARCHITECTURE

Application loading configurations run optimization processes within development builders to keep interactions responsive across slower networks:

- **Dynamic Component Isolation:** Key workspace modules use lazy-loading strategies to split route components, preventing dashboard views from processing heavy editor assets prematurely.
- **Client Event Filtering:** Text fields process inputs through 500ms debounce mechanisms, shielding central store components from unnecessary re-rendering work during active typing.
- **Asset File Vectorization:** Template previews display using lightweight embedded SVG vector paths instead of standard raw image files, eliminating sudden layout shifts during theme adjustments.

SECTION 13 — SECURITY REVIEW

The application isolates internal account pages from public anonymous access using multi-tier route guards enforced at the client layer:

- **Authorization Header Controls:** Security validation keys are stored in client memory. Global Axios interceptors inject bearer keys automatically into outgoing backend requests.
- **Role Verification Guards:** Administrative paths wrap inside specialized checking layers. These intercept unauthorized requests and route invalid sessions back to account dashboards.
- **Input Validation Standards:** Form components filter out malicious code sequences using input formatting rules built into validation frameworks, shielding client runtimes from cross-site scripting attempts.

SECTION 14 — FIGMA FILE STRUCTURE

To support systematic handover workflows, the project design library organizes file spaces matching development directory paths:

```
❖ CareerForge Pro Master Design Ecosystem v3.0
├─ ↳ ❖ Page 01: Typography Engine Scales & Baseline Element Layers
├─ ↳ ❖ Page 02: Primitive Variables & Semantic Token Palettes
├─ ↳ ❖ Page 03: Atomic Presentation Primitives (Buttons, Input Fields)
├─ ↳ ❖ Page 04: Public Portal Screens [SCR-01 to SCR-03]
├─ ↳ ❖ Page 05: Authenticated Workspace Modules [SCR-04 to SCR-06]
```

SECTION 15 — DESIGN REVIEW CHECKLIST

Product teams verify component alignment parameters against this quality control index prior to production releases:

- **Senior Designers:** Verify canvas token variables transition accurately between layout modes without dropping contrast compliance metrics.
- **Lead Architects:** Ensure route protection guards intercept unauthenticated requests and block layout processing loops cleanly.
- **Frontend Engineers:** Confirm data collection components apply input debouncing to minimize unnecessary store recalculations.

SECTION 16 — IMPLEMENTATION ROADMAP

System production schedules distribute development tasks across four specialized milestone periods:

- **Phase 1 (Milestone 1-2):** Build atomic presentation elements and establish design token variables across common layout libraries.
- **Phase 2 (Milestone 3-4):** Implement protected navigation components and set up data distribution slices inside store engines.
- **Phase 3 (Milestone 5-6):** Assemble side-by-side split layout areas, wiring fields directly to live client validation frameworks.
- **Phase 4 (Milestone 7-8):** Connect administrative management tools and run accessibility evaluations ahead of product deployment.

SECTION 17 — FINAL ARCHITECTURE SCORECARD

Evaluation metrics summarize structural readiness characteristics across seven specialized engineering categories:

- **Scalability Index (9.4 / 10):** Decoupling data states from styling properties allows teams to grow template libraries smoothly.
- **Maintainability Score (9.2 / 10):** Structuring layouts by feature folders minimizes cross-module code dependencies.
- **Accessibility Rating (9.6 / 10):** Robust field indicators paired with focus trap boundaries satisfy target accessibility standards.
- **Performance Metric (9.0 / 10):** Route-based component splitting maintains fast initialization speeds over slower connection lines.